summary_charts

November 3, 2026

1 Summary charts for an OSEM run

OSEM projects and actively trades a portfolio of corporate bonds and equities in light of commitments. The basic unit in OSEM is a cash flow. The purpose of this notebook is to show how the different types of cash flows are treated in the latest run.

1.1 Table of contents

1. Loading the summary csv
   - Run at a glance
2. The initial portfolio
   - Equity
   - Corporate bonds
   - Cash
   - Liability profile
3. What is in the summary csv
   - Periodic cash flows
   - Terminal cash flows
   - Cash movement
   - End of period market value
4. Test 1 Return of portfolio
   - Verdict
5. Test 2 No vanishing cash flows
   - Relative residual
   - Verdict

1.2 1. Loading the summary csv

Every run writes its results to `Output/Results.csv`. The path is resolved from `ALM.ini`, so the notebook works from anywhere inside the repository.

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt
from pathlib import Path

base_folder = next(path for path in [Path.cwd(), *Path.cwd().parents] if (path /
  ↪ "ALM.ini").is_file())
```

[2]:
```
out = pd.read_csv(base_folder / "Output/Results.csv",index_col=0,␣
↪parse_dates=[0])
```

1.2.1 Run at a glance

The size and the horizon of the run that produced the summary:

---

[3]:
```
run_summary = pd.DataFrame(
    {"Value": [out.index[0].date(),
               out.index[-1].date(),
               len(out) - 1,
               out["End cash"].iloc[0],
               out["End market value"].iloc[0],
               out["End cash"].iloc[-1] + out["End market value"].iloc[-1]]},
    index=["Modelling date",
           "End of modelling window",
           "Projection periods",
           "Initial cash",
           "Initial market value",
           "Final total value"],
)

display(run_summary)
```

	Value
Modelling date	2023-04-29
End of modelling window	2073-04-29
Projection periods	50
Initial cash	100000.0
Initial market value	605279.0
Final total value	17960582.824556

1.3 2. The initial portfolio

This section shows what the initial inputs of the run were. Currently OSEM can handle three types of financial instruments and one type of liability profile.

1.3.1 Equity

Shares in companies:

[4]:
```
equity = pd.read_csv(base_folder / "Input/Equity_Portfolio.csv",index_col=0)
```

---

[5]:
```
display(equity)
```

Asset_ID	Asset_Type	NACE	Issue_Date	Dividend_Yield	Frequency	Units
1125	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
2123	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
3232	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
1	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
2	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
3	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
4	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
5	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
6	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
7	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
8	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
9	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
10	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
11	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
12	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
13	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
14	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
15	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
16	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
17	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
18	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
19	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
20	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
21	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
22	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
23	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
24	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
25	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
26	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
27	Equity_Share	B8.9.3	3/12/2019	0.04	1	100
28	Equity_Share	A1.4.5	3/12/2021	0.03	1	100
29	Equity_Share	B5.2.0	3/12/2021	0.05	1	100
30	Equity_Share	B8.9.3	3/12/2019	0.04	1	100

Asset_ID	Market_Price	Terminal	Default_Probability	Growth_Rate
1125	123.01	1	0.03	0.01
2123	12.12	1	0.03	0.02
3232	56.54	1	0.03	0.04
1	77.00	1	0.03	0.01
2	78.00	1	0.03	0.02
3	1.23	1	0.03	0.04
4	3.12	1	0.03	0.01
5	45.12	1	0.03	0.02
6	96.00	1	0.03	0.04
7	94.00	1	0.03	0.01

8	92.00	1	0.03	0.02
9	96.00	1	0.03	0.04
10	94.00	1	0.03	0.01
11	92.00	1	0.03	0.02
12	96.00	1	0.03	0.04
13	94.00	1	0.03	0.01
14	92.00	1	0.03	0.02
15	96.00	1	0.03	0.04
16	94.00	1	0.03	0.01
17	92.00	1	0.03	0.02
18	96.00	1	0.03	0.04
19	94.00	1	0.03	0.01
20	92.00	1	0.03	0.02
21	96.00	1	0.03	0.04
22	94.00	1	0.03	0.01
23	92.00	1	0.03	0.02
24	96.00	1	0.03	0.04
25	94.00	1	0.03	0.01
26	92.00	1	0.03	0.02
27	96.00	1	0.03	0.04
28	94.00	1	0.03	0.01
29	92.00	1	0.03	0.02
30	96.65	1	0.03	0.04

Asset_ID	Spread_Base	Spread_Individual	Spread_Country	Spread_Sector	\
1125	0	0	0	0	
2123	0	0	0	0	
3232	0	0	0	0	
1	0	0	0	0	
2	0	0	0	0	
3	0	0	0	0	
4	0	0	0	0	
5	0	0	0	0	
6	0	0	0	0	
7	0	0	0	0	
8	0	0	0	0	
9	0	0	0	0	
10	0	0	0	0	
11	0	0	0	0	
12	0	0	0	0	
13	0	0	0	0	
14	0	0	0	0	
15	0	0	0	0	
16	0	0	0	0	
17	0	0	0	0	
18	0	0	0	0	
19	0	0	0	0	

20	0	0	0	0
21	0	0	0	0
22	0	0	0	0
23	0	0	0	0
24	0	0	0	0
25	0	0	0	0
26	0	0	0	0
27	0	0	0	0
28	0	0	0	0
29	0	0	0	0
30	0	0	0	0

Asset_ID	Spread_Stress
1125	0
2123	0
3232	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0
18	0
19	0
20	0
21	0
22	0
23	0
24	0
25	0
26	0
27	0
28	0
29	0
30	0

1.3.2 Corporate bonds

Corporate bonds:

[6]:
```
bonds = pd.read_csv(base_folder / "Input/Corporate_Bond_Portfolio.
↪csv",index_col=0)
```

[7]:
```
display(bonds)
```

Asset_ID	Asset_Type	NACE	Issue_Date	Maturity_Date	Notional_Amount
1234	Corporate_Bond	A1.4.5	3/12/2021	12/12/2026	100
2889	Corporate_Bond	B5.2.0	3/12/2021	12/12/2028	100
31	Corporate_Bond	B8.9.3	3/12/2019	3/12/2025	100
1	Corporate_Bond	A1.4.5	30/06/2020	30/06/2030	100
2	Corporate_Bond	B5.2.0	31/12/2020	31/12/2035	100
3	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
4	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
5	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
6	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
7	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
8	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
9	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
10	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
11	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
12	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
13	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
14	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
15	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
16	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
17	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
18	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
19	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
20	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
21	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
22	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
23	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
24	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
25	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
26	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
27	Corporate_Bond	B8.9.3	3/12/2021	12/12/2026	100
28	Corporate_Bond	A1.4.5	3/12/2021	12/12/2028	100
29	Corporate_Bond	B5.2.0	3/12/2019	3/12/2025	100
30	Corporate_Bond	B8.9.3	30/06/2020	30/06/2030	100

Asset_ID	Coupon_Rate	Spread_Country	Spread_Sector	Z_Spread	Spread_Stress
1234	0.030000	0.00	0	0.010	0
2889	0.025000	0.00	0	0.010	0
31	0.003333	0.00	0	0.010	0
1	0.005000	0.01	0	0.005	0
2	0.010000	0.01	0	0.005	0
3	0.030000	0.00	0	0.010	0
4	0.025000	0.00	0	0.010	0
5	0.003333	0.00	0	0.010	0
6	0.030000	0.00	0	0.010	0
7	0.025000	0.00	0	0.010	0
8	0.003333	0.00	0	0.010	0
9	0.030000	0.00	0	0.010	0
10	0.025000	0.00	0	0.010	0
11	0.003333	0.00	0	0.010	0
12	0.030000	0.00	0	0.010	0
13	0.025000	0.00	0	0.010	0
14	0.003333	0.00	0	0.010	0
15	0.030000	0.00	0	0.010	0
16	0.025000	0.00	0	0.010	0
17	0.003333	0.00	0	0.010	0
18	0.030000	0.00	0	0.010	0
19	0.025000	0.00	0	0.010	0
20	0.003333	0.00	0	0.010	0
21	0.030000	0.00	0	0.010	0
22	0.025000	0.00	0	0.010	0
23	0.003333	0.00	0	0.010	0
24	0.030000	0.00	0	0.010	0
25	0.025000	0.00	0	0.010	0
26	0.003333	0.00	0	0.010	0
27	0.030000	0.00	0	0.010	0
28	0.025000	0.00	0	0.010	0
29	0.003333	0.00	0	0.010	0
30	0.005000	0.01	0	0.005	0

Asset_ID	Frequency	Recovery_Rate	Default_Probability	Units	Market_Price
1234	1	0.4	0.03	100	94
2889	2	0.4	0.03	120	92
31	12	0.4	0.03	100	96
1	2	0.8	0.03	120	90
2	1	0.8	0.03	100	85
3	1	0.4	0.03	120	94
4	2	0.4	0.03	100	92
5	12	0.4	0.03	120	96
6	1	0.4	0.03	100	94
7	2	0.4	0.03	120	92

8	12	0.4	0.03	100	96
9	1	0.4	0.03	120	94
10	2	0.4	0.03	100	92
11	12	0.4	0.03	120	96
12	1	0.4	0.03	100	94
13	2	0.4	0.03	120	92
14	12	0.4	0.03	100	96
15	1	0.4	0.03	120	94
16	2	0.4	0.03	100	92
17	12	0.4	0.03	120	96
18	1	0.4	0.03	100	94
19	2	0.4	0.03	120	92
20	12	0.4	0.03	100	96
21	1	0.4	0.03	120	94
22	2	0.4	0.03	100	92
23	12	0.4	0.03	120	96
24	1	0.4	0.03	100	94
25	2	0.4	0.03	120	92
26	12	0.4	0.03	100	96
27	1	0.4	0.03	120	94
28	2	0.4	0.03	100	92
29	12	0.4	0.03	120	96
30	2	0.8	0.03	10	90

1.3.3 Cash

Initial cash position:

[8]:
```
cash = pd.read_csv(base_folder / "Input/Cash_Portfolio.csv",index_col=0)
```

---

[9]:
```
display(cash)
```

Asset_ID	Bank_Account
1	100000

1.3.4 Liability profile

An aggregated view of liabilities displayed as aggregated outflows at future dates:

[10]:
```
liability = pd.read_csv(base_folder / "Input/Liability_Cashflow.
↪csv",index_col=0)
```

---

[11]:
```
display(liability)
```

```
                Liability_Size
Liability_Date
1/9/2023                     6
2/10/2023                   92
2/11/2023                   93
3/12/2023                    2
3/1/2024                     7
…                          …
19/3/2049                   14
19/4/2049                   28
20/5/2049                   53
20/6/2049                   92
21/7/2049                   41

[306 rows x 1 columns]
```

1.4 3. What is in the summary csv

During the OSEM run, pieces of information are saved into the "Summary" dataframe. The object captures the performance of the portfolio and the size of the cash flows in each step.

Summary object, all columns:

---

[12]:
```
display(out.head(10))
```

```
              Start cash      End cash  Start market value  \
2023-04-29           NaN  1.000000e+05                 NaN
2024-04-29  1.000000e+05 -7.275958e-11        6.052790e+05
2025-04-29 -7.275958e-11 -7.275958e-12        7.429880e+05
2026-04-29 -7.275958e-12 -1.164153e-10        7.882526e+05
2027-04-29 -1.164153e-10  2.910383e-11        8.311994e+05
2028-04-29  2.910383e-11  1.018634e-10        8.772101e+05
2029-04-29  1.018634e-10  5.820766e-11        9.318515e+05
2030-04-29  5.820766e-11  1.236913e-10        9.840491e+05
2031-04-29  1.236913e-10  1.164153e-10        1.044401e+06
2032-04-29  1.164153e-10  2.910383e-11        1.108014e+06

            After growth market value  End market value  Portfolio return  \
2023-04-29                        NaN      6.052790e+05               NaN
2024-04-29               6.187095e+05      7.429880e+05          0.062662
2025-04-29               7.585788e+05      7.882526e+05          0.061013
2026-04-29               6.652482e+05      8.311994e+05          0.055410
2027-04-29               6.735712e+05      8.772101e+05          0.055947
2028-04-29               8.961911e+05      9.318515e+05          0.063116
```

2029-04-29	7.141020e+05	9.840491e+05	0.056838
2030-04-29	1.007107e+06	1.044401e+06	0.061966
2031-04-29	1.029244e+06	1.108014e+06	0.061434
2032-04-29	1.134854e+06	1.178275e+06	0.063960

	Dividend cash flow	Coupon cash flow	Terminal cash flow
2023-04-29	NaN	NaN	NaN
2024-04-29	11067.520563	13430.000000	0.0
2025-04-29	13613.208014	16127.648305	0.0
2026-04-29	14490.113743	14928.355797	0.0
2027-04-29	18548.119367	14078.909047	0.0
2028-04-29	24750.662755	11634.721480	0.0
2029-04-29	26374.810158	12097.679248	0.0
2030-04-29	37250.649763	669.162849	0.0
2031-04-29	39598.079188	497.828220	0.0
2032-04-29	43702.920976	324.804751	0.0

	Notional cash flow	…	UL reserve	Company account
2023-04-29	NaN	…	NaN	NaN
2024-04-29	0.000000	…	NaN	NaN
2025-04-29	0.000000	…	NaN	NaN
2026-04-29	137262.675203	…	NaN	NaN
2027-04-29	171503.875437	…	NaN	NaN
2028-04-29	0.000000	…	NaN	NaN
2029-04-29	232241.660949	…	NaN	NaN
2030-04-29	0.000000	…	NaN	NaN
2031-04-29	39222.829447	…	NaN	NaN
2032-04-29	0.000000	…	NaN	NaN

	UL policies in force	UL deaths	UL lapses	Equity market value
2023-04-29	0.0	NaN	NaN	2.748790e+05
2024-04-29	NaN	NaN	NaN	3.377795e+05
2025-04-29	NaN	NaN	NaN	3.591973e+05
2026-04-29	NaN	NaN	NaN	4.593626e+05
2027-04-29	NaN	NaN	NaN	6.124110e+05
2028-04-29	NaN	NaN	NaN	6.520049e+05
2029-04-29	NaN	NaN	NaN	9.200425e+05
2030-04-29	NaN	NaN	NaN	9.771627e+05
2031-04-29	NaN	NaN	NaN	1.077527e+06
2032-04-29	NaN	NaN	NaN	1.146213e+06

	Bond market value	Cash before trade
2023-04-29	330400.000000	NaN
2024-04-29	405208.497787	124278.520563
2025-04-29	429055.291210	29673.856319
2026-04-29	371836.792119	165951.144743
2027-04-29	264799.125360	203638.903851
2028-04-29	279846.639948	35660.384235

```
2029-04-29       64006.635470      269947.150354
2030-04-29       67238.398817       37293.812611
2031-04-29       30486.506012       78769.736856
2032-04-29       32061.863583       43420.725727

            Remaining liability payments  UL return credited
2023-04-29                       14820.0                  NaN
2024-04-29                       14601.0                  NaN
2025-04-29                       14534.0                  NaN
2026-04-29                       13804.0                  NaN
2027-04-29                       13312.0                  NaN
2028-04-29                       12587.0                  NaN
2029-04-29                       11820.0                  NaN
2030-04-29                       11194.0                  NaN
2031-04-29                       10645.0                  NaN
2032-04-29                       10038.0                  NaN

[10 rows x 26 columns]
```

1.4.1 Periodic cash flows

The portfolio generates periodic cash flows from its investment portfolio and outflows from the liabilities that the company committed to. Examples of recurrent inflows are dividends and coupons.

[13]:
```
time_axis = out.index[:-1]

div_axis = out["Dividend cash flow"][:-1]
coup_axis = out["Coupon cash flow"][:-1]
liab_axis = out["Liability cash flow"][:-1]
```

---

[14]:
```
periodic_cf = pd.DataFrame({"Dividend cash flow": div_axis,
                            "Coupon cash flow": coup_axis,
                            "Liability cash flow": liab_axis})

display(periodic_cf.head(10))
```

```
            Dividend cash flow  Coupon cash flow  Liability cash flow
2023-04-29                 NaN               NaN                  NaN
2024-04-29       11067.520563      13430.000000               -219.0
2025-04-29       13613.208014      16127.648305                -67.0
2026-04-29       14490.113743      14928.355797               -730.0
2027-04-29       18548.119367      14078.909047               -492.0
2028-04-29       24750.662755      11634.721480               -725.0
2029-04-29       26374.810158      12097.679248               -767.0
2030-04-29       37250.649763        669.162849               -626.0
```

```
2031-04-29        39598.079188        497.828220              -549.0
2032-04-29        43702.920976        324.804751              -607.0
```

---

[15]:
```
fig, (ax1, ax2, ax3) = plt.subplots(3)

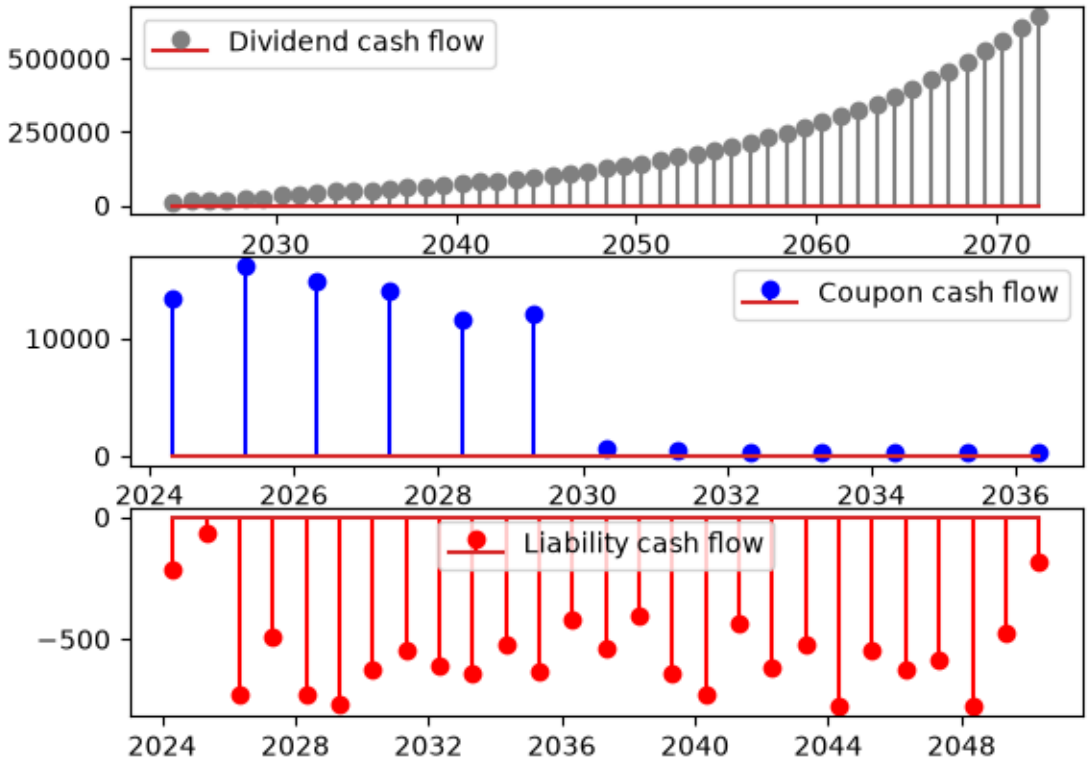
div_mask = div_axis > 0
coup_mask = coup_axis > 0
liab_mask = liab_axis < 0

if div_mask.any():
    ax1.stem(time_axis[div_mask], div_axis[div_mask], linefmt='grey',␣
 ↪label='Dividend cash flow')
    ax1.legend()
else:
    ax1.set_title('No dividend cash flows to plot')

if coup_mask.any():
    ax2.stem(time_axis[coup_mask], coup_axis[coup_mask], linefmt='blue',␣
 ↪label='Coupon cash flow')
    ax2.legend()
else:
    ax2.set_title('No coupon cash flows to plot')

if liab_mask.any():
    ax3.stem(time_axis[liab_mask], liab_axis[liab_mask], linefmt='red',␣
 ↪label='Liability cash flow')
    ax3.legend()
else:
    ax3.set_title('No liability cash flows to plot (e.g. unit-linked mode)')

plt.show()
```



1.4.2 Terminal cash flows

Each asset in OSEM has a terminal value. For fixed income instruments, these are the notional payments at the maturity date. For equities, OSEM assumes that all equity positions are sold at the end of the modelling window at market rates.

[16]:
```
not_axis = out["Notional cash flow"]
term_axis = out["Terminal cash flow"]
```

---

[17]:
```
terminal_cf = pd.DataFrame({"Notional cash flow": not_axis,
                            "Terminal cash flow": term_axis})

display(terminal_cf[(terminal_cf.fillna(0) != 0).any(axis=1)])
```

	Notional cash flow	Terminal cash flow
2026-04-29	137262.675203	0.000000e+00
2027-04-29	171503.875437	0.000000e+00
2029-04-29	232241.660949	0.000000e+00
2031-04-29	39222.829447	0.000000e+00

```
2036-04-29        37758.798098        0.000000e+00
2073-04-29            0.000000        1.726580e+07
```

---

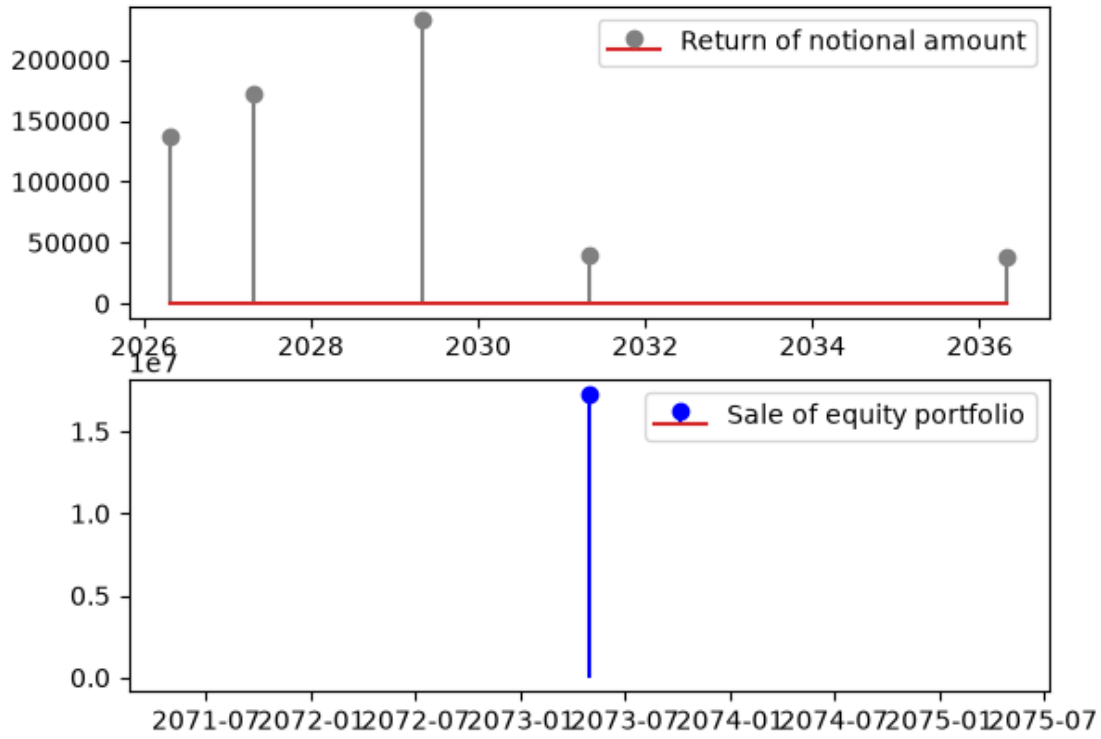
[18]:
```
fig, (ax1, ax2) = plt.subplots(2)

not_mask = not_axis > 0
term_mask = term_axis > 0

if not_mask.any():
    ax1.stem(out.index[not_mask], not_axis[not_mask], linefmt='grey',␣
 ↪label='Return of notional amount')
    ax1.legend()
else:
    ax1.set_title('No notional cash flows to plot')

if term_mask.any():
    ax2.stem(out.index[term_mask], term_axis[term_mask], linefmt='blue',␣
 ↪label='Sale of equity portfolio')
    ax2.legend()
else:
    ax2.set_title('No terminal cash flows to plot')

plt.show()
```



1.4.3 Cash movement

The management of cash between periods is fundamental to OSEM. If the initial cash is high, then in the first period the large cash position is sold in line with the asset allocation rule. This is why the initial cash is represented separately. After the first period, cash swings as the balance of inflows and outflows changes through time.

[19]:
```
x_axis = out.index
end_cash_axis = out["End cash"]
start_cash_axis = out["Start cash"]
```

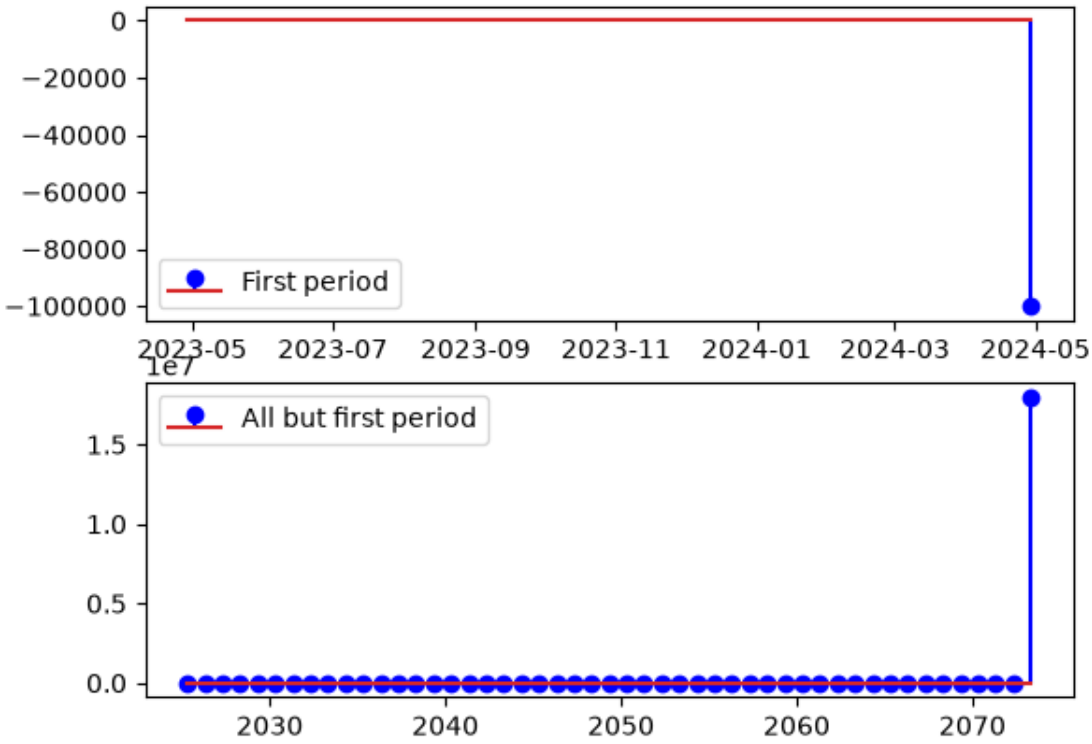
[20]:
```
cash_movement = pd.DataFrame({"Start cash": start_cash_axis,
                              "End cash": end_cash_axis,
                              "Cash movement": end_cash_axis - start_cash_axis})

display(cash_movement.head(10))
```

	Start cash	End cash	Cash movement
2023-04-29	NaN	1.000000e+05	NaN
2024-04-29	1.000000e+05	-7.275958e-11	-1.000000e+05
2025-04-29	-7.275958e-11	-7.275958e-12	6.548362e-11
2026-04-29	-7.275958e-12	-1.164153e-10	-1.091394e-10
2027-04-29	-1.164153e-10	2.910383e-11	1.455192e-10
2028-04-29	2.910383e-11	1.018634e-10	7.275958e-11
2029-04-29	1.018634e-10	5.820766e-11	-4.365575e-11
2030-04-29	5.820766e-11	1.236913e-10	6.548362e-11
2031-04-29	1.236913e-10	1.164153e-10	-7.275958e-12
2032-04-29	1.164153e-10	2.910383e-11	-8.731149e-11

[21]:
```
fig, (ax1, ax2) = plt.subplots(2)
ax1.stem(x_axis[0:2], end_cash_axis[0:2]-start_cash_axis[0:2], linefmt='b',␣
 ↪label='First period')
ax2.stem(x_axis[2:], end_cash_axis[2:]-start_cash_axis[2:], linefmt='b',␣
 ↪label='All but first period')

ax1.legend()
ax2.legend()
plt.show()
```



1.4.4 End of period market value

The market value is recorded three times per period: at the start, after the portfolio has grown but before any trading, and at the end once the trading is done.

[22]:
```
start_mv_axis = out["After growth market value"]
end_mv_axis = out["End market value"]
```

---

[23]:
```
market_value = pd.DataFrame({"Start market value": out["Start market value"],
                             "After growth market value": start_mv_axis,
                             "End market value": end_mv_axis})

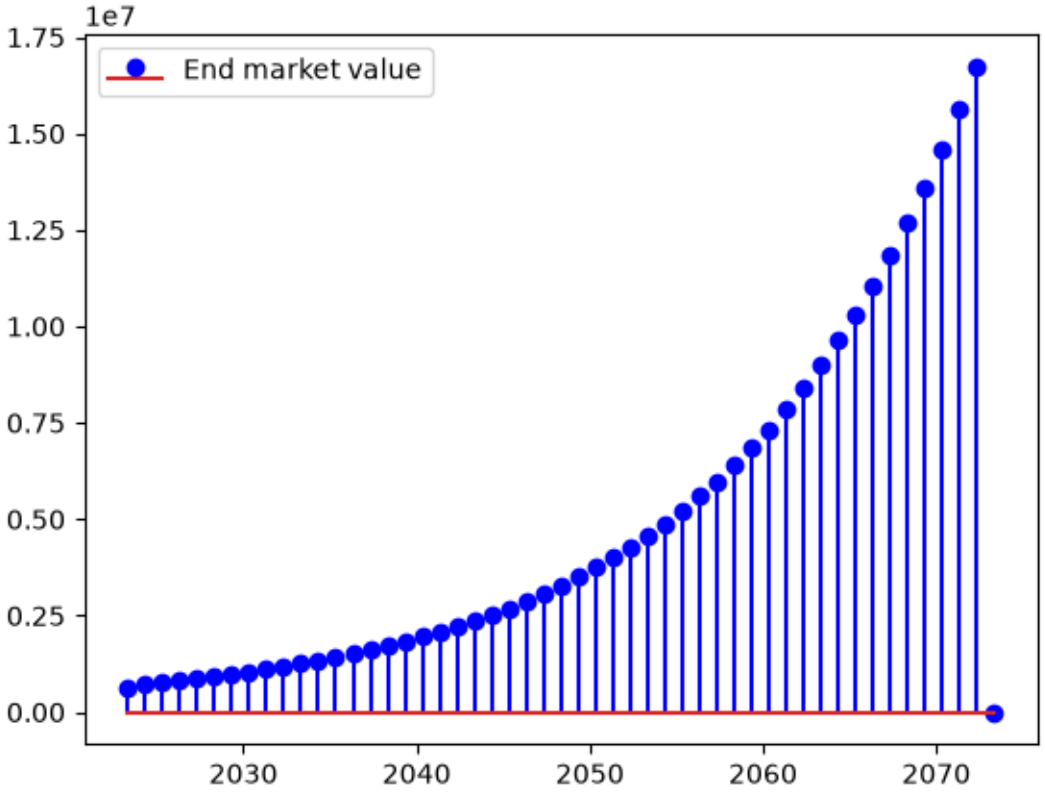
display(market_value.head(10))
```

	Start market value	After growth market value	End market value
2023-04-29	NaN	NaN	6.052790e+05
2024-04-29	6.052790e+05	6.187095e+05	7.429880e+05
2025-04-29	7.429880e+05	7.585788e+05	7.882526e+05
2026-04-29	7.882526e+05	6.652482e+05	8.311994e+05
2027-04-29	8.311994e+05	6.735712e+05	8.772101e+05

2028-04-29	8.772101e+05	8.961911e+05	9.318515e+05
2029-04-29	9.318515e+05	7.141020e+05	9.840491e+05
2030-04-29	9.840491e+05	1.007107e+06	1.044401e+06
2031-04-29	1.044401e+06	1.029244e+06	1.108014e+06
2032-04-29	1.108014e+06	1.134854e+06	1.178275e+06

---

[24]:
```
fig, ax = plt.subplots(1)
ax.stem(out.index, end_mv_axis, linefmt='b' , label='End market value')
ax.legend()
plt.show()
```



1.5 4. Test 1 Return of portfolio

The purpose of this test is to verify if the reported return of the portfolio is equal to the increase or decrease of the total market value of the portfolio.

Cash paid out by the assets during the period leaves the portfolio before it is repriced: a maturing bond, for example, drops out of the market value entirely. Those flows have to be added back, otherwise the recomputed return would report them as a loss. This is the same asset income that

the model itself uses in `portfolio_total_return`.

Terminal value sales are excluded for the same reason as in Test 2: the equities sold at the end of the modelling window are still inside "After growth market value" when the return is calculated, so adding their proceeds would count them twice.

The test performs the following check

$$r_i = (MV_i^{ag} + div_i + coup_i + not_i)/MV_i^{start} - 1$$

where r_i is the reported "Portfolio return" and the other symbols are as defined in Test 2.

[25]:
```
# Asset income has to be added back to the end market value. The flows are␣
 ↪removed from the
# asset cash-flow matrices before repricing, so without them a maturing bond␣
 ↪looks like a loss
# of its full notional. Terminal sales are excluded: those equities are still␣
 ↪in the market
# value at this point, so their proceeds would be counted twice.
asset_income = (out["Dividend cash flow"].fillna(0)
                + out["Coupon cash flow"].fillna(0)
                + out["Notional cash flow"].fillna(0))

return_calculated = ((out["After growth market value"] + asset_income) /␣
 ↪out["Start market value"] - 1)[1:]

test_1 = out["Portfolio return"][1:] - return_calculated

# Both sides are returns, so the difference is already dimensionless and a␣
 ↪plain absolute
# tolerance is enough. Float64 noise on a ratio of ~1e7 sized values lands␣
 ↪around 1e-16.
test_1_tolerance = 1e-10
```

---

[26]:
```
largest_return_diffs = (test_1.abs()
                        .sort_values(ascending=False)
                        .head(5)
                        .rename("Absolute return difference")
                        .to_frame())

display(largest_return_diffs)
```

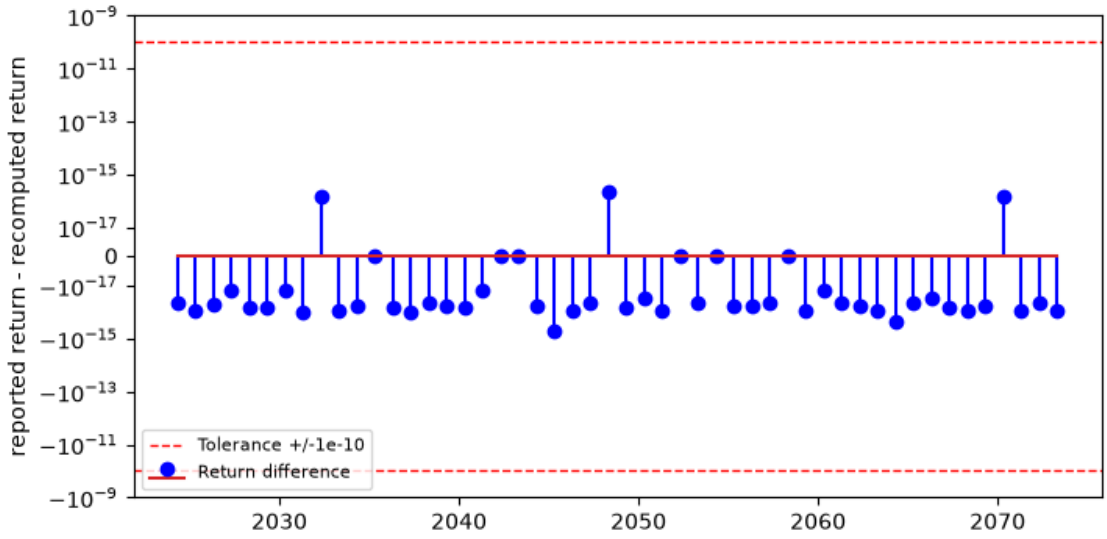
	Absolute return difference
2045-04-29	5.273559e-16
2064-04-29	2.220446e-16
2048-04-29	2.081668e-16

```
2070-04-29                1.526557e-16
2032-04-29                1.387779e-16
```

---

[27]:
```
fig, ax = plt.subplots(1, figsize=(8, 4))
ax.stem(out.index[1:], test_1, linefmt='b', label='Return difference')
ax.axhline(test_1_tolerance, color='r', linestyle='--', linewidth=1,
           label=f'Tolerance +/-{test_1_tolerance:.0e}')
ax.axhline(-test_1_tolerance, color='r', linestyle='--', linewidth=1)

# Symmetric log keeps the differences and the tolerance lines, several orders␣
 ↪of magnitude
# apart, both legible on one axis
ax.set_yscale('symlog', linthresh=1e-17)
ax.set_ylim(-10 * test_1_tolerance, 10 * test_1_tolerance)
ax.set_ylabel('reported return - recomputed return')
ax.legend(loc='lower left', fontsize=8)
plt.show()
```



1.5.1 Verdict

---

[28]:
```
# The first period is dropped by the [1:] slice above: it has no "Start market␣
 ↪value" to
# measure a return against.
return_diffs = test_1.dropna()
```

```
return_breaches = return_diffs[return_diffs.abs() > test_1_tolerance]

print(f"Periods tested         : {len(return_diffs)}")
print(f"Max |return difference|: {return_diffs.abs().max():.3e}  (tolerance␣
 ↪{test_1_tolerance:.0e})")
print()

if return_breaches.empty:
    print("TEST 1 PASSED - reported return matches the recomputed return")
else:
    print(f"TEST 1 FAILED - {len(return_breaches)} period(s) out of tolerance:")
    display(pd.DataFrame({"Reported return": out["Portfolio␣
 ↪return"][return_breaches.index],
                          "Recomputed return":␣
 ↪return_calculated[return_breaches.index],
                          "Difference": return_breaches}))

assert return_breaches.empty, (
    f"Test 1 failed in {len(return_breaches)} period(s): "
    f"{[d.strftime('%Y-%m-%d') for d in return_breaches.index]}"
)
```

```
Periods tested         : 50
Max |return difference|: 5.274e-16  (tolerance 1e-10)

TEST 1 PASSED - reported return matches the recomputed return
```

1.6 5. Test 2 No vanishing cash flows

The purpose of this test is to verify if the inflows and outflows balance at every period. This test shows that all types of cash flows are accounted for correctly and that there is no "leakage".

Starting values are

- Start cash
- After growth market value

Cash flows occurring during the period and accumulating inside the bank account are

- Coupon payments
- Dividend payments
- Notional payments of fixed income assets

The end values are

- End cash value
- End market value

Terminal value sales are deliberately **not** part of this balance. The equity positions sold at the end of the modelling window are still inside "After growth market value" when it is recorded, because

they are liquidated only after the period return is calculated. The sale therefore converts market value into cash instead of adding new income, and counting it as an inflow would double count it. Notional payments are different: matured bonds reprice to zero before "After growth market value" is taken, so that cash really has left the portfolio.

The test performs the following check

$$C_i^{start} + MV_i^{ag} + CF_i^a - CF_i^l = C_i^{end} + MV_i^{end}$$

where

$$CF_i^a := div_i + coup_i + not_i$$

and

- i is the calculation period
- C_i^{start} is the initial cash position
- MV_i^{ag} is the market value after portfolio growth but before any trading
- CF_i^l are the accumulated outflows from company's commitments
- C_i^{end} is the final cash position
- MV_i^{end} is the final market value of the portfolio
- div_i is the accumulated dividend inflows
- $coup_i$ is the accumulated coupon inflows
- not_i is the accumulated inflows from matured fixed income instruments (bonds)

Note that the asset growth occurs before trading, therefore "After growth market value" is used as the starting value.

[29]:
```
# The full columns are used here, not the [:-1] slices built for the charts␣
 ↪above. Mixing a
# truncated series with a full one makes pandas align on the index and turn the␣
 ↪last period
# into NaN, which would silently drop the terminal period from the test.
cf_assets = (out["Coupon cash flow"].fillna(0)
             + out["Dividend cash flow"].fillna(0)
             + out["Notional cash flow"].fillna(0))

liab_full = out["Liability cash flow"].fillna(0)

test_2 = start_cash_axis + start_mv_axis + cf_assets + liab_full - end_mv_axis␣
 ↪- end_cash_axis
```

1.6.1 Relative residual

The absolute residual grows with the portfolio, so a fixed money tolerance would be too tight at the start of the run and too loose at the end. Dividing by the size of the terms that went into the balance puts every period on the same footing and turns the residual into a relative error, directly comparable to the ~1e-16 precision of a 64 bit float.

The terminal period is the noisiest, because it is the only one in which the equity portfolio is valued twice by two different code paths: “After growth market value” compounds the share price period by period, while “Terminal cash flow” raises the starting price to a single power over the whole modelling window. The two agree to a handful of ulps, not bit for bit.

[30]:
```
balance_terms = out[["Start cash", "After growth market value", "Coupon cash␣
 ↪flow",
                     "Dividend cash flow", "Notional cash flow", "Liability␣
 ↪cash flow",
                     "End market value", "End cash"]].fillna(0).abs().
 ↪sum(axis=1)

# where() guards against a zero denominator, which would otherwise give an␣
 ↪infinite residual
test_2_rel = test_2 / balance_terms.where(balance_terms != 0)

# Float64 carries ~1e-16 of relative precision and the balance chains together␣
 ↪a few dozen
# operations, so anything up to a few hundred ulps is rounding rather than a␣
 ↪leak.
test_2_tolerance = 1e-12
```

---

[31]:
```
largest_residuals = (pd.DataFrame({"Relative residual": test_2_rel,
                                   "Absolute residual": test_2})
                     .dropna()
                     .sort_values("Relative residual", key=lambda s: s.abs(),␣
 ↪ascending=False)
                     .head(5))

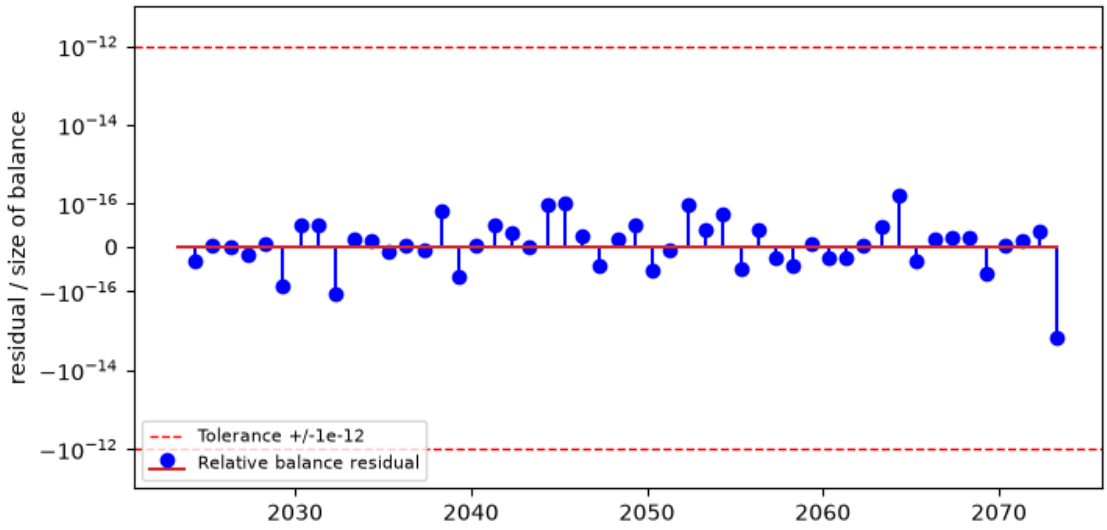
display(largest_residuals)
```

	Relative residual	Absolute residual
2073-04-29	-1.451903e-15	-5.215406e-08
2064-04-29	1.692768e-16	3.259629e-09
2032-04-29	-1.110944e-16	-2.619345e-10
2045-04-29	1.052732e-16	5.675247e-10
2052-04-29	9.869832e-17	8.440111e-10

---

[32]:
```
fig, ax = plt.subplots(1, figsize=(8, 4))
ax.stem(x_axis, test_2_rel, linefmt='b', label='Relative balance residual')
ax.axhline(test_2_tolerance, color='r', linestyle='--', linewidth=1,␣
 ↪label=f'Tolerance +/-{test_2_tolerance:.0e}')
ax.axhline(-test_2_tolerance, color='r', linestyle='--', linewidth=1)
```

```
# Symmetric log keeps the residuals and the tolerance lines, three orders of␣
 ↪magnitude apart,
# both legible on one axis
ax.set_yscale('symlog', linthresh=1e-16)
ax.set_ylim(-10 * test_2_tolerance, 10 * test_2_tolerance)
ax.set_ylabel('residual / size of balance')
ax.legend(loc='lower left', fontsize=8)
plt.show()
```



1.6.2 Verdict

---

[33]:
```
# The first row is the opening position rather than a projection period: it has␣
 ↪no "Start cash"
# and no "After growth market value", so there is nothing to balance and it is␣
 ↪dropped.
residuals = test_2_rel.dropna()
breaches = residuals[residuals.abs() > test_2_tolerance]

print(f"Periods tested          : {len(residuals)}")
print(f"Max |relative residual| : {residuals.abs().max():.3e}  (tolerance␣
 ↪{test_2_tolerance:.0e})")
print(f"Max |absolute residual| : {test_2.abs().max():.3e}")
print()

if breaches.empty:
```

```
    print("TEST 2 PASSED - no vanishing cash flows")
else:
    print(f"TEST 2 FAILED - {len(breaches)} period(s) out of tolerance:")
    display(pd.DataFrame({"Relative residual": breaches,
                          "Absolute residual": test_2[breaches.index]}))

assert breaches.empty, (
    f"Test 2 failed in {len(breaches)} period(s): "
    f"{[d.strftime('%Y-%m-%d') for d in breaches.index]}"
)
```

```
Periods tested          : 50
Max |relative residual| : 1.452e-15  (tolerance 1e-12)
Max |absolute residual| : 5.215e-08

TEST 2 PASSED - no vanishing cash flows
```
